In [2]:
```
import tensorflow as tf
import matplotlib.pyplot as plt
from tensorflow import keras
from tensorflow.keras import layers
import pandas as pd
```

In [5]:
```
train_data = pd.read_csv("mnist_train.csv")
test_data = pd.read_csv("mnist_test.csv")

print("Training Data : " , train_data.shape)
print("Test Data : " , test_data.shape)
```

```
Training Data :  (60000, 785)
Test Data :  (10000, 785)
```

In [6]:
```
# Separate labels and pixel values
y_train = train_data.iloc[:, 0].values
y_test = test_data.iloc[:, 0].values

x_train = train_data.iloc[:, 1:].values
x_test = test_data.iloc[:, 1:].values

# Check the shapes
print("Training images:", x_train.shape)
print("Training labels:", y_train.shape)

print("Testing images:", x_test.shape)
print("Testing labels:", y_test.shape)
```

```
Training images: (60000, 784)
Training labels: (60000,)
Testing images: (10000, 784)
Testing labels: (10000,)
```

In [7]:
```
x_train = x_train / 255.0
x_test = x_test / 255.0

print("Minimum pixel value:", x_train.min())
print("Maximum pixel value:", x_train.max())
```

```
Minimum pixel value: 0.0
Maximum pixel value: 1.0
```

In [9]:
```
model = keras.Sequential([
    keras.Input(shape=(784,)),
    layers.Flatten(),
    layers.Dense(128, activation='relu'),
    layers.Dense(10, activation='softmax')
])

model.summary()
```

**Model: "sequential_1"**

| Layer (type) | Output Shape | Param # |
|---|---|---|
| flatten_1 (Flatten) | (None, 784) | 0 |
| dense_2 (Dense) | (None, 128) | 100,480 |
| dense_3 (Dense) | (None, 10) | 1,290 |

**Total params:** 101,770 (397.54 KB)

**Trainable params:** 101,770 (397.54 KB)

**Non-trainable params:** 0 (0.00 B)

In [10]:
```
model.compile(
    optimizer='SGD',
    loss='sparse_categorical_crossentropy',
    metrics=['accuracy']
)
```

```
WARNING:tensorflow:TensorFlow GPU support is not available on native Windows for TensorFlow >= 2.11. Even if CUDA/cuDNN are ins
talled, GPU will not be used. Please use WSL2 or the TensorFlow-DirectML plugin.
```

In [11]:
```
history = model.fit(
    x_train,
    y_train,
    epochs=11,
```

```
    batch_size=32
)
```

```
Epoch 1/11
1875/1875 ━━━━━━━━━━━━━━━━━━━━ 4s 2ms/step - accuracy: 0.8358 - loss: 0.6454
Epoch 2/11
1875/1875 ━━━━━━━━━━━━━━━━━━━━ 3s 2ms/step - accuracy: 0.9051 - loss: 0.3380
Epoch 3/11
1875/1875 ━━━━━━━━━━━━━━━━━━━━ 4s 2ms/step - accuracy: 0.9183 - loss: 0.2897
Epoch 4/11
1875/1875 ━━━━━━━━━━━━━━━━━━━━ 3s 2ms/step - accuracy: 0.9281 - loss: 0.2596
Epoch 5/11
1875/1875 ━━━━━━━━━━━━━━━━━━━━ 3s 2ms/step - accuracy: 0.9338 - loss: 0.2371
Epoch 6/11
1875/1875 ━━━━━━━━━━━━━━━━━━━━ 3s 2ms/step - accuracy: 0.9395 - loss: 0.2191
Epoch 7/11
1875/1875 ━━━━━━━━━━━━━━━━━━━━ 3s 2ms/step - accuracy: 0.9431 - loss: 0.2042
Epoch 8/11
1875/1875 ━━━━━━━━━━━━━━━━━━━━ 4s 2ms/step - accuracy: 0.9473 - loss: 0.1914
Epoch 9/11
1875/1875 ━━━━━━━━━━━━━━━━━━━━ 4s 2ms/step - accuracy: 0.9498 - loss: 0.1799
Epoch 10/11
1875/1875 ━━━━━━━━━━━━━━━━━━━━ 4s 2ms/step - accuracy: 0.9528 - loss: 0.1702
Epoch 11/11
1875/1875 ━━━━━━━━━━━━━━━━━━━━ 4s 2ms/step - accuracy: 0.9550 - loss: 0.1615
```

In [12]:
```
test_loss, test_accuracy = model.evaluate(x_test, y_test)

print("Test Loss:", test_loss)
print("Test Accuracy:", test_accuracy)
```

```
313/313 ━━━━━━━━━━━━━━━━━━━━ 1s 2ms/step - accuracy: 0.9547 - loss: 0.1564
Test Loss: 0.15643447637557983
Test Accuracy: 0.9546999931335449
```

In [13]:
```
import matplotlib.pyplot as plt

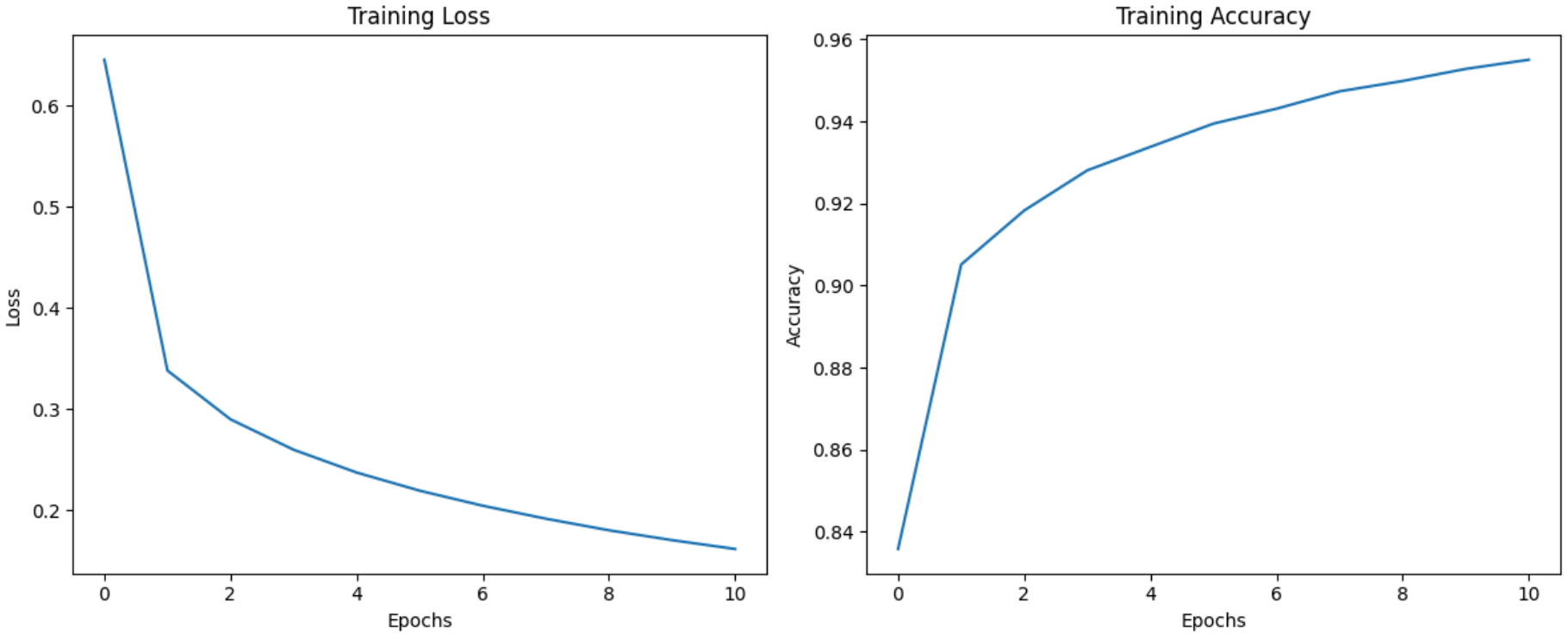
# Plot training loss
plt.figure(figsize=(12, 5))

plt.subplot(1, 2, 1)
```

```
plt.plot(history.history['loss'])
plt.title('Training Loss')
plt.xlabel('Epochs')
plt.ylabel('Loss')

# Plot training accuracy
plt.subplot(1, 2, 2)
plt.plot(history.history['accuracy'])
plt.title('Training Accuracy')
plt.xlabel('Epochs')
plt.ylabel('Accuracy')

plt.tight_layout()
plt.show()
```




In [ ]: